

JUSTIFICACIÓN DE ESTRUCTURAS DE DATOS

Sistema de Gestión Académica y Planificación de Rutas de Aprendizaje

Curso	Estructuras de Datos
Programa	Ingeniería de Sistemas
Docente	Ing. Jhon Haide Cano Beltran.
Estudiante	Steven Zuñiga Monsalve
Fecha	Mayo 2026

1. Introducción

El presente documento justifica las decisiones de implementación tomadas durante el desarrollo del Sistema de Gestión Académica. Para cada estructura de datos utilizada se explica: qué es, por qué se eligió sobre otras alternativas, cuál es su complejidad (Big-O) y cómo se usa concretamente en el sistema.

El enunciado permitía elegir libremente entre implementar las estructuras manualmente o usar las clases provistas por Java, siempre que la decisión quedara justificada. A continuación se detalla cada decisión.

2. Decisión General de Implementación

Se tomó la siguiente decisión global para el proyecto:

Tipo de estructura	Implementación elegida	Razón
Matrices y arreglos	Arreglos nativos de Java	Obligatorio según enunciado
Listas enlazadas	LinkedList de Java	Justificado en sección 3.3
Pilas	Stack de Java	Justificado en sección 3.4
Colas	ArrayDeque de Java	Justificado en sección 3.5
HashMap / TreeMap	Clases de Java	Obligatorio según enunciado

3. Justificación por Estructura

3.1 HashMap — Índice de estudiantes

¿Qué es?

Un mapa de clave-valor que internamente usa una tabla hash. Permite acceder a cualquier elemento directamente por su clave sin recorrer toda la colección.

¿Por qué HashMap y no un arreglo?

Con un arreglo, buscar un estudiante por ID requeriría recorrerlo completo en el peor caso: $O(n)$. Con HashMap, el acceso es $O(1)$ en promedio gracias al hash de la clave. En un sistema con cientos de estudiantes, esto marca una diferencia significativa en rendimiento.

Complejidad:

```
Insertar: O(1) promedio Buscar: O(1) promedio Eliminar: O(1) promedio
```

Uso en el sistema:

```
HashMap estudiantes; estudiantes.put(id, nuevoEstudiante); // registrar
estudiantes.get(id); // buscar estudiantes.remove(id); // eliminar
```

3.2 TreeMap — Aulas ordenadas

¿Qué es?

Un mapa que mantiene sus claves ordenadas automáticamente. Internamente usa un árbol rojo-negro (árbol binario de búsqueda balanceado).

¿Por qué TreeMap y no HashMap?

El enunciado requiere que las aulas estén ordenadas por nombre. HashMap no garantiza orden. TreeMap ordena las claves alfabéticamente de forma automática sin necesidad de implementar lógica adicional de ordenamiento.

Complejidad:

```
Insertar: O(log n) Buscar: O(log n) Recorrer: O(n) en orden alfabetico
garantizado
```

Uso en el sistema:

```
TreeMap aulas; // Al iterar, las aulas aparecen en orden alfabetico
automaticamente
```

3.3 LinkedList — Pre-requisitos e historial

¿Qué es?

Una lista doblemente enlazada donde cada nodo contiene un dato y dos referencias: al nodo anterior y al siguiente. No tiene tamaño fijo.

¿Por qué LinkedList y no implementación propia?

La LinkedList de Java ya implementa correctamente la estructura de nodos enlazados internamente. Implementarla manualmente hubiera requerido crear la clase Nodo y todos los métodos (add, remove, contains, iterator), aumentando el riesgo de errores sin aportar diferencia funcional en el comportamiento del sistema. La clase de Java está optimizada, probada y produce el mismo resultado que una implementación propia.

¿Por qué LinkedList y no ArrayList?

Los pre-requisitos se agregan de forma dinámica y no se accede a ellos por índice numérico — solo se recorren para verificar. LinkedList es más eficiente para inserciones y eliminaciones que ArrayList en este caso.

Complejidad:

```
Agregar al final: O(1) Eliminar: O(n) Recorrer: O(n)
```

Uso en el sistema:

```
LinkedList preRequisitos; // en Materia LinkedList historialMaterias; // en
Estudiante preRequisitos.add(codigoMateria); historial.contains(codigo); //
verificar si va curso la materia
```

3.4 Stack — Deshacer/Rehacer y navegación

¿Qué es?

Una pila LIFO (Last In, First Out): el último elemento en entrar es el primero en salir. En Java, Stack extiende Vector e implementa push/pop/peek.

¿Por qué Stack y no implementación propia?

La funcionalidad de deshacer/rehacer requiere exactamente el comportamiento LIFO que Stack provee. Implementarla manualmente con un arreglo o lista enlazada hubiera requerido gestionar manualmente el índice del tope y los casos borde (pila vacía, pila llena). Stack de Java encapsula todo eso correctamente y permite enfocarse en la lógica del negocio.

Complejidad:

```
push (apilar): O(1) pop (desapilar): O(1) peek (ver tope): O(1) isEmpty: O(1)
```

Uso en el sistema:

```
Stack pilaDeshacer; Stack pilaRehacer; Stack pilaReportes; // navegacion entre
reportes pilaDeshacer.push(new Object[]{"INSCRIBIR", idEst, codMat}); Object[]
op = pilaDeshacer.pop(); // deshacer
```

3.5 ArrayDeque (Queue) — Cola de espera y Batch

¿Qué es?

Una cola FIFO (First In, First Out) implementada con un arreglo circular redimensionable. En Java se usa como Queue a través de la interfaz Queue.

¿Por qué ArrayDeque y no LinkedList como cola?

Aunque LinkedList también implementa Queue, ArrayDeque es más eficiente en memoria porque no crea objetos Nodo para cada elemento — usa un arreglo interno. Para colas de alta frecuencia como el procesamiento batch, ArrayDeque es la opción recomendada por la propia documentación de Java.

¿Por qué cola (FIFO) y no pila (LIFO)?

En una cola de espera para materias, la justicia exige que el primer estudiante en llegar sea el primero en obtener el cupo. FIFO garantiza esto. Una pila daría ventaja al último en llegar, lo cual sería incorrecto.

Complejidad:

```
add (encolar): O(1) amortizado poll (desencolar): O(1) peek (ver frente): O(1)
isEmpty: O(1)
```

Uso en el sistema:

```
Queue colaEspera; // en Materia Queue colaSolicitudes; // en ProcesadorBatch
colaEspera.add(idEstudiante); // agregar a espera colaEspera.poll(); //
asignar cupo al primero
```

3.6 Arreglos y Matrices Nativas — Obligatorio

boolean[7][24] — Disponibilidad horaria de aulas

Representa los 7 días de la semana y las 24 horas del día. El acceso directo por índice `horario[dia][hora]` es $O(1)$, lo que permite verificar y reservar horarios de forma instantánea sin recorrer estructuras.

int[N][N] — Matriz de adyacencia entre edificios

Representa las distancias entre N edificios. El algoritmo de Dijkstra requiere acceso directo entre pares de nodos: `matrizDistancias[i][j]`. Una matriz de adyacencia es la representación natural para grafos densos y permite que Dijkstra funcione en $O(V^2)$ donde V es el número de edificios.

Double[10][20] — Notas por semestre

Almacena hasta 10 semestres con 20 materias cada uno. El uso de `Double` (wrapper) en lugar de `double` (primitivo) permite distinguir entre una nota de 0.0 y una posición vacía (`null`), lo cual es fundamental para calcular promedios correctamente ignorando materias no cursadas.

Facultad[5] — Lista fija de facultades

Arreglo de tamaño fijo porque las facultades de una universidad no cambian dinámicamente en el sistema. Un arreglo nativo es la estructura más simple y eficiente para una colección de tamaño conocido y fijo.

4. Resumen de Decisiones

Estructura	Implementación	Justificación clave
HashMap	Java	Busqueda $O(1)$ por ID de estudiante
TreeMap	Java	Orden alfabético automático de aulas
LinkedList	Java	Insertion dinámica, misma funcionalidad que propia
Stack	Java	LIFO nativo para deshacer/rehacer
ArrayDeque	Java	FIFO eficiente para colas de espera y batch
boolean[7][24]	Nativa	Obligatorio - acceso $O(1)$ por día y hora
int[N][N]	Nativa	Obligatorio - soporte para Dijkstra $O(V^2)$
Double[10][20]	Nativa	Obligatorio - distingue <code>null</code> de 0.0
Facultad[5]	Nativa	Obligatorio - tamaño fijo conocido

